

A Comprehensive Framework for EURUSD Optimal Trade Entry Prediction: From Adaptive Labeling to Regime-Aware Model Evaluation

*Quantitative ML Pipeline Research
EURUSD Algorithmic Trading System*

May 2026

Keywords: Optimal Trade Entry, EURUSD, Temporal Convolutional Networks, LSTM, XGBoost, Triple-Barrier Labeling, Regime Detection, Walk-Forward Backtesting, CUSUM Filter, Threshold Calibration, Financial Machine Learning

Abstract

This paper presents a comprehensive end-to-end machine learning framework for predicting Optimal Trade Entry (OTE) points in the EURUSD currency pair on five-minute intrabar data. The framework addresses the full pipeline from raw OHLCV ingestion through adaptive label generation, multi-family feature engineering, feature selection, sequence model training, and regime-aware policy backtesting. Drawing on over 2.5 million five-minute bars spanning 2010 to 2026, the system generates three families of structural trade labels—reversals, continuation-pullbacks, and compression-to-expansion breakouts—each constructed with volatility-adaptive thresholds, Triple Barrier outcome validation, trend-scan statistical cross-validation, and Lopez de Prado-style purging and sample-uniqueness weighting.

Feature engineering produces over 150 candidate signals organized across momentum, volatility, volume, cycle, statistical, session, and ICT-derived categories. Feature importance is computed via a merged ranking pipeline combining random forest Mean Decrease in Impurity (MDI), mutual information, SHAP mean absolute values, and SHAP positive-class attribution scores, with sequential attribution gates filtering to the 90th cumulative-importance percentile before hyperparameter optimization begins.

Model training employs purged walk-forward cross-validation across XGBoost (with sparse geometric lag windows), Temporal Convolutional Networks (TCN) with causal dilated residual blocks, and Long Short-Term Memory (LSTM) sequence classifiers. Hyperparameters are optimized via Optuna TPE sampling with MedianPruner across 12-20 trials, and probability outputs are calibrated with either Platt scaling or isotonic regression. Post-training evaluation operates through a deterministic regime-labeling layer defining composite regimes as the cross-product of five trend states (ranging, weak_up, weak_down, strong_up, strong_down) with three volatility states (low, medium, high), yielding 15 distinct regime cells evaluated independently. Threshold calibration is performed per-regime using event-based F-beta scores with $\beta=0.55$ to weight precision over recall, supplemented by a regime-conditional abstain policy covering high-stress market conditions, session spread costs, and cooldown enforcement.

Walk-forward policy backtesting across eight or more quarterly folds with 40-bar purge gaps produces trade-level equity curves, Walk-Forward Efficiency (WFE) ratios, monthly Sharpe ratios, profit factors, and annualized net pips after a session-spread cost model. The post-training leaderboard of 19 unique model identifiers across 63 saved backtest runs shows that reversal models significantly outperform breakout models in live-like evaluation despite breakout models dominating the training-phase leaderboard by CV average precision. The strongest performing models—short_reversal_xgb_v2 (monthly Sharpe 3.61, profit factor 2.74) and long_reversal_tcn_v3 (annualized net pips 3,558, profit factor 2.58)—passed all six acceptance gates and demonstrate the superiority of regime-conditional evaluation over raw CV metrics for practitioner deployment decisions.

1. Introduction

1.1 The Challenge of EURUSD Prediction

The EURUSD currency pair is the world's most liquid financial instrument, with daily notional turnover exceeding 1.5 trillion USD. This liquidity, paradoxically, makes prediction structurally harder rather than easier. At the five-minute timeframe, price movements are dominated by microstructure noise, institutional order flow imbalances, and calendar-driven volatility clustering that overwhelms any single-factor signal. The pair exhibits non-stationary volatility regimes—alternating between sub-pip Asian consolidation and 30-50 pip London open expansions within the same 24-hour period—making fixed-horizon return forecasting statistically unreliable without explicit conditioning on market state.

The classical academic approach of regressing future returns on lagged indicators fails in this setting for at least three reasons. First, the signal-to-noise ratio at short prediction horizons is extremely low; any apparent predictability is rapidly arbitrated away by algorithmic market participants. Second, the non-stationarity of regime transitions means that a model trained on trending periods will actively generate losses when applied to ranging conditions. Third, and most critically for machine learning, the sequential autocorrelation structure of financial time series violates the i.i.d. assumption underlying standard train-test splits, meaning that naive cross-validation dramatically overestimates out-of-sample performance.

Beyond these statistical challenges, the practitioner faces an economic constraint that the academic literature routinely ignores: transaction costs. At realistic EURUSD spreads of 1.0-1.5 pips during London session and 2.5 pips during Asian hours, any strategy must generate gross returns that comfortably exceed the round-trip cost before it has any economic value. A model with 55% accuracy but small expected moves will systematically lose money once spreads and slippage are accounted for.

1.2 Why Standard ML Fails in Financial Markets

The application of standard machine learning methods—random forests, gradient boosting, recurrent networks trained with i.i.d. mini-batches—to financial time series prediction has been extensively documented to produce overfit models with negligible out-of-sample value. Several structural pathologies explain this failure. Look-ahead contamination is the most insidious: features computed from future prices leak predictive information backward in time, creating spuriously high in-sample accuracy that evaporates in real-time deployment. Label bias is equally problematic; using simple future-return labels does not capture the asymmetric risk-reward structure of trading, where the timing and magnitude of adverse excursions matter as much as the direction of the move.

The overlap problem, formalized by Lopez de Prado (2018) in 'Advances in Financial Machine Learning', describes how position-overlapping labels—where the outcome window of one observation extends into the feature window of the next—create correlated training samples that bias gradient estimators and inflate cross-validation scores. Standard k-fold cross-validation on sequential data produces test sets that are chronologically embedded within the training period, allowing the model to effectively 'see' future information through the autocorrelation structure of the features.

Class imbalance presents a further challenge: high-quality trade entry signals are rare by construction. In a well-calibrated labeling system, positive labels represent perhaps 1-3% of all bars, making naive accuracy a degenerate metric and requiring specialized loss functions and evaluation criteria. The standard approach of maximizing accuracy is not only theoretically inappropriate but actively harmful: it encourages the model to predict the majority class (no-trade) with high confidence while ignoring the minority class that contains all the economic value.

1.3 Contributions of This Framework

This paper documents a framework that addresses each of these failure modes through an integrated pipeline of methodological choices. The primary contributions are: (1) a three-

family adaptive labeling system using structurally-meaningful ATR thresholds derived from the 1-hour timeframe rather than the prediction timeframe, preventing consolidation clustering in the label set; (2) a Triple Barrier outcome validation layer that filters labels by economic viability rather than price proximity alone; (3) a trend-scan statistical cross-validation step using ordinary least squares t-statistics over multiple forward windows to confirm directional momentum at each labeled event; (4) a purged walk-forward cross-validation regime with explicit bar-level embargo buffers between training and validation folds; (5) a composite regime taxonomy derived from ADX, EMA alignment, ATR percentile rank, and session classification that conditions both threshold selection and abstain policy on market state; and (6) a post-training evaluation framework that prioritizes post-cost trade-level profitability over in-sample average precision.

The gap between academic machine learning and practical trading system design is most visible in the disconnect between training-phase leaderboards and post-training deployment outcomes. This framework explicitly measures this gap across 42 model artifacts and 63 saved backtest runs, demonstrating that the highest-CV-AP models (breakout family) are post-cost unprofitable in walk-forward simulation, while mid-table training performers (reversal family) consistently pass all six acceptance gates with Sharpe ratios exceeding 2.7 and profit factors exceeding 2.5. This dissociation motivates the framework's emphasis on post-training evaluation as the primary promotion criterion.

2. Data and Preprocessing

2.1 Data Sources and Coverage

The primary dataset consists of EURUSD five-minute OHLCV bars sourced from institutional FX data providers covering the period 2010 to 2026, yielding approximately 2.5 million bars after preprocessing. The five-minute timeframe is the primary resolution for both labeling and feature computation. Higher-timeframe views—30-minute and 1-hour bars—are computed by deterministic resampling using open=first, high=max, low=min, close=last, volume=sum aggregation, implemented in the `resample_ohlc()` function in `labeling_engine.py`. This ensures strict causality: the resampled bars at any point in time are computed only from the five-minute bars that have fully closed.

The `load_fx_csv()` function in `labeling_engine.py` handles multi-format data ingestion, resolving date/time columns from both separate Date/Time column pairs (format '%Y%m%d %H:%M:%S') and unified timestamp columns. Column normalization maps provider-specific aliases—'vol', 'tick_volume', 'o', 'h', 'l', 'c', 'adj close'—to the canonical lowercase OHLCV schema. Timezone normalization via `normalize_datetime_series()` from `features/fx_calendar.py` converts source timestamps (configurable, default UTC) to a canonical storage timezone (default UTC), with explicit metadata tracking of the conversion to prevent downstream timezone drift.

2.2 Anomaly Detection and Data Quality

The `detect_anomalies()` function in `labeling_engine.py` implements a three-condition anomaly mask. A bar is flagged and removed if any of the following conditions hold: (1) structural OHLC invalidity, meaning $high < low$, $close > high$, or $close < low$; (2) a bar range exceeding 10x the rolling 20-bar ATR, which captures data feed errors and flash-crash artifacts; (3) zero or negative volume, which indicates a synthetic fill or data provider gap. Anomalous bars are removed before any labeling or feature computation proceeds. The count of removed bars is tracked in the dataset metadata as `anomaly_count` and propagated through the pipeline.

Duplicate timestamps are resolved by keeping the first occurrence, and the dataset is then sorted chronologically. The pipeline also supports date range filtering via `filter_date_range()` for selective retraining on sub-periods, with inclusive-to-exclusive semantics for date-only boundaries to capture full trading days. Source row counts before and after each filtering stage are tracked in DataFrame attrs for downstream metadata reporting.

3. Target Labeling Methodology

3.1 Design Philosophy: Structural vs. Fixed-Horizon Labels

The labeling methodology departs fundamentally from fixed-horizon and simple directional labels that dominate the academic financial ML literature. Fixed-horizon labels assign a class based on the sign of the return n bars ahead, which conflates entry timing with holding period determination and assigns the same label quality to a clean swing entry and a noisy mid-range entry. The triple-barrier method, introduced by Lopez de Prado (2018, Ch. 3.4), improves on this by conditioning the label on which barrier—profit target, stop loss, or time limit—is hit first, but still does not capture the structural quality of the entry point itself.

The framework's labeling engines are event-driven rather than bar-driven. Labels are generated from detected structural events—swing highs, swing lows, breakout confirmations, and pullback completions—and then projected onto the 5-minute bar grid. This means that most bars carry a negative (no-event) label, and positive labels mark the specific bars where the structural condition was met. This design choice creates an inherently imbalanced dataset that better reflects the economic reality of trading: profitable opportunities arise rarely and specifically, not uniformly in time.

A critical innovation in this framework is the use of 1-hour ATR as the 'structural ATR' for all threshold computations in the reversal labeler, and 30-minute ATR for the continuation and breakout labelers. This replaces the naive approach of using 5-minute ATR (approximately 1 pip for EURUSD) with a structurally-meaningful volatility measure (20-30 pips for EURUSD on the 1-hour timeframe). The function `map_hrf_atr_to_ltf()` in `reversal_labeling_engine.py` implements this via causal forward-fill: the higher-timeframe ATR value is applied to all lower-timeframe bars until the next higher-timeframe bar closes, ensuring no look-ahead contamination.

3.2 Reversal Labeling Engine

3.2.1 CUSUM Filter

The reversal labeling pipeline (`reversal_labeling_engine.py`) begins with a CUSUM filter, adapted from Lopez de Prado (2018, Ch. 2.5.2), to identify structurally significant price movements before computing swing points. The symmetric CUSUM filter maintains two accumulators: `s_pos` for upward cumulative log-returns and `s_neg` for downward, each initialized to zero. At each bar, the log-return $\log(P_t / P_{t-1})$ is added to both accumulators (clamped to zero on the opposite side):

$$s_{pos}(t) = \max(0, s_{pos}(t-1) + r_t) \quad s_{neg}(t) = \min(0, s_{neg}(t-1) + r_t)$$

An event is flagged when $s_{pos} \geq h$ or $s_{neg} \leq -h$, where h is the dynamic threshold $h = \text{cusum_atr_mult} \times \text{structural_ATR}(t)$, with `cusum_atr_mult`=0.5 as the default. This adaptive threshold ensures that the sensitivity of the CUSUM filter scales with prevailing volatility: periods of high structural ATR require larger price movements to trigger events, preventing over-triggering in volatile regimes. The `cusum_use_dynamic` flag controls whether the dynamic threshold or a fixed `cusum_fixed_threshold`=0.0020 is used.

3.2.2 Zigzag Swing Detection with Structural ATR

The `detect_swings_zigzag()` function implements a causal state-machine swing detector with three key improvements over naive zigzag algorithms. The detector tracks a direction state variable (1 for seeking highs, -1 for seeking lows) and candidate swing price/index variables. Confirmation of a swing high requires the close price to retrace by at least `confirm_atr_mult` $\times$ `structural_ATR` below the candidate high (`confirm_atr_mult`=0.8, corresponding to approximately 18-24 pips of retrace in EURUSD at typical 1-hour ATR levels). This prevents trivial swings in consolidation from qualifying.

Beyond the retrace confirmation, three additional quality filters are enforced. First, the minimum bar distance between consecutive swings (`minBarsBetweenSwings`=18, equivalent to 90 minutes at 5m) prevents rapid-fire detections in choppy markets. Second,

the minimum price distance filter (`min_swing_distance_atr=0.75`) requires consecutive swings to be separated by at least 0.75x the structural ATR in price space; this prevents consolidation clustering where many small swings cluster near the same price level. Third, the minimum swing size filter (`min_swing_atr=1.35`) requires that the full amplitude of the swing—from the previous confirmed swing to the current candidate—exceed 1.35x the structural ATR. Swings that pass all three filters are recorded as `SwingPoint` dataclass instances with full metadata including `source_tf`, `confirm_lag`, `atr_at_swing`, and `swing_size_atr`.

3.2.3 Triple Barrier Outcome Validation

Following swing detection, each candidate swing point is validated through the Triple Barrier Method (Lopez de Prado, 2018, Ch. 3.4), implemented in `validate_swings_triple_barrier()`. For each confirmed swing low (candidate long entry), the method sets a profit target at:

$$TP = entry + tb_profit_atr \times structural_ATR \quad SL = entry - tb_stop_atr \times structural_ATR$$

with `tb_profit_atr=1.0` and `tb_stop_atr=2.0` as defaults (asymmetric 1:2 risk-reward). The method then scans forward up to `tb_maxBars=120` bars (10 hours at 5-minute resolution), recording which barrier is hit first: 'tp' (profit target reached), 'sl' (stop loss triggered), or 'timeout' (neither hit within the maximum holding period). Swings with `tb_outcome='sl'` are excluded from final labels via the `is_positive_swing()` gate, which enforces the criterion that only setups where the subsequent price action validated the directional hypothesis are used as positive training examples. Timeout outcomes receive a `tb_score` of 0.45 in the quality computation, reflecting their ambiguous economic character.

3.2.4 Trend-Scan Statistical Cross-Validation

Each surviving swing point is further validated using the `trend_scan_swings()` function, which computes ordinary least squares t-statistics of log-price against time over multiple forward windows. For a window of length `L` bars starting at the swing index, the OLS t-statistic on the slope coefficient `beta_1` in:

$$\log(P_{\{t+k\}}) = \beta_0 + \beta_1 * k + \epsilon_k, \quad k = 0..L$$

is computed via `_linear_trend_tstat()`. For swing lows, the signed t-statistic is `t_signed = t_raw` (positive slope expected); for swing highs, `t_signed = -t_raw` (negative slope expected). The `trend_scan_windows` tuple=(12, 24, 48, 96) spans 1 through 8 hours of forward price action, and the maximum signed t-statistic across all windows is recorded. A swing passes trend-scan validation if this maximum exceeds `trend_scan_min_abs_t=2.0`, indicating statistically significant directional momentum following the swing. The `trend_scan_enforce` flag (default True) gates label assignment on passing this test.

3.2.5 Label Quality Scoring and Tiering

The `compute_swing_quality()` function assigns a continuous quality score to each validated swing point as a weighted combination of four components:

$$\text{label_quality} = 0.35 * \text{tb_score} + 0.30 * \text{trend_score} + 0.20 * \text{htf_score} + 0.15 * \text{entry_score}$$

where `tb_score` maps Triple Barrier outcomes {tp->1.0, timeout->0.45, skip->0.35, sl->0.0}; `trend_score` = `clip(trend_scan_t / 4.0, 0, 1)`; `htf_score` = `0.5 * htf_match_30m + 0.5 * htf_match_1h`; and `entry_score` = `clip(entry_score / 5.0, 0, 1)`. HTF matching (`htf_match_30m`, `htf_match_1h`) indicates whether the 5-minute swing occurred within a configurable time window—default 120 minutes for 30m, 240 minutes for 1h—of a coincident higher-timeframe swing, providing multi-timeframe confluence validation. Swings are tiered as A (quality >= 0.80), B (>= 0.65), C (>= min_label_quality = 0.35), or reject.

3.2.6 Sample Uniqueness Weights

To mitigate the double-counting problem that arises when overlapping label windows assign correlated training examples to the same observation, the `compute_sample_weights()` function implements inverse-concurrency weighting following Lopez de Prado (2018, Ch. 4.4). For each bar, `compute_label_concurrency()` counts the number of active label windows covering that bar. The sample weight is then:

$$w(t) = 1 / \max(\text{concurrency}(t), 1)$$

This weight is further modulated by the `label_quality` score, ensuring that high-quality events receive proportionally larger gradient contributions. The final `sample_weight` column used in model training is: $w(t) * \max(\text{label_quality}(t), \text{min_label_quality})$ for positive labels, and 1.0 for negative labels.

3.2.7 Exclusion Zones and Purge Buffers

The `create_exclusion_masks()` function implements Lopez de Prado's purging concept (Ch. 7.1) by creating exclusion zones of `exclusion_preBars=10` bars before each confirmation point. Bars within these zones are excluded from the negative training class even if they carry a negative label, preventing the model from learning to distinguish positive from negative using information that is temporally adjacent to the event—a form of near-lookahead bias. The `neg_ok_long_reversal` and `neg_ok_short_reversal` boolean columns indicate which negative bars are safe to use as training negatives.

3.3 Continuation-Pullback Labeling Engine

The continuation-pullback labeler (`ote_continuation_pullback_labeling_engine.py`) targets a fundamentally different setup: shallow retracements within an established higher-timeframe trend that subsequently resume the trend direction. The `ContinuationPullbackParams` dataclass specifies a tighter parameter regime: `structural_atr_tf='30m'` (rather than 1hr), `confirm_atr_mult=0.45` (vs 0.8 for reversals), `min_swing_atr=0.70`, `min_swing_distance_atr=0.35`, and `min_bars_between_swings=8`.

The `build_trend_context()` function computes EMA-21/EMA-50 trend direction on both 30-minute and 1-hour bars, mapping these to the 5-minute bar index via causal forward-fill. A 30-minute EMA spread is defined as $(\text{EMA_fast} - \text{EMA_slow}) / \text{ATR_30m}$; `trend_up_30m` requires this spread to exceed `min_trend_spread_atr=0.10` ATR units, and `require_1h_trend_alignment=True` enforces that the 1-hour trend must agree with the 30-minute trend direction before a continuation label can be assigned. The `filter_continuation_candidates()` function then screens swing lows (in uptrends) and swing highs (in downtrends) by additional criteria: pullback depth must fall within `[pullback_min_atr=0.18, pullback_max_atr=2.00]` structural ATR units, the number of

pullback bars must not exceed `max_pullbackBars=48`, and the price at pullback confirmation must not have violated the prior trend floor by more than `trend_floor_tolerance_atr=0.50` ATR units.

Triple Barrier parameters for continuation setups are tighter than for reversals: `tb_profit_atr=0.80`, `tb_stop_atr=0.70`, `tb_max_bars=36`. This reflects the expectation that continuation trades resolve faster and with narrower stops than reversal trades, which require more room for the directional thesis to develop.

3.4 Breakout Labeling Engine

The breakout labeler (`ote_breakout_labeling_engine.py`) targets compression-to-expansion patterns: bars that close above a lookback range high or below a lookback range low following a period of narrow consolidation. The `BreakoutParams` dataclass uses `structural_atr_tf='30m'`, and the `detect_breakout_events()` function scans each bar against a `prior_range_high` and `prior_range_low` computed from a rolling `breakout_lookback_bars=20` window with a one-bar lag to prevent look-ahead contamination.

A long breakout is detected when five conditions are met simultaneously: (1) the previous close was below the range high plus a buffer (`breakout_close_buffer_atr=0.05` ATR), preventing detection of bars that were already above the range; (2) the current bar's high reaches or exceeds the range high; (3) the current bar's close exceeds the range high by at least the buffer, confirming the break on a closing basis; (4) the breakout extension— $(\text{close} - \text{range_high}) / \text{ATR}$ —does not exceed `max_breakout_extension_atr=0.80`, filtering out late entries that have already captured most of the move; and (5) bar quality conditions—`body_fraction >= breakout_min_body_fraction=0.25` and `close_location_in_bar >= breakout_min_close_location=0.65`—confirm that the bar showed decisive directional commitment rather than a tail spike.

Compression validation (`breakout_requires_compression=True` by default) enforces that the compression range—computed from `compression_lookback_bars=12` rolling high

minus low—must not exceed `compression_max_range_atr=1.80` structural ATR units before the breakout, distinguishing genuine consolidation breakouts from random price penetrations of loose ranges.

3.5 Combined Labeling Pipeline and Label Statistics

The three labeling engines are orchestrated by `build_combined_label_frame()` in `labeling_engine.py`, which runs all three in parallel on the same 5-minute dataset and merges their outputs into a single bar-level DataFrame. Each family contributes independent label columns: `label_long_{family}`, `label_short_{family}`, `label_long_{family}_entry`, `label_short_{family}_entry`, `label_quality_long_{family}`, and `sample_weight_long_{family}`. A shared `warmup_mask` column excludes the first `warmupBars=50` bars from all label statistics.

Table 1. Label family summary statistics and key parameters.

Label Family	Positive Rate (Long)	Positive Rate (Short)	TP Rate (approx)	Key Parameters
Reversal	~1-2%	~1-2%	45-55%	<code>confirm_atr=0.8</code> , <code>min_swing_atr=1.35</code> , <code>tb_max=120</code>
Continuation	~2-4%	~2-4%	50-60%	<code>confirm_atr=0.45</code> , <code>min_swing_atr=0.70</code> , <code>tb_max=36</code>
Breakout	~1-3%	~1-3%	35-45%	<code>confirm_atr=0.25</code> , <code>min_swing_atr=0.50</code> , <code>tb_max=24</code>

The `summarize_labels()` function computes per-family positive counts, entry counts, and mean quality scores. The event CSV export via `combine_event_frames()` preserves full metadata for each labeled event including Triple Barrier outcomes, trend-scan t-statistics, HTF confluence flags, entry scores, entry risk-reward ratios, and label quality scores, enabling downstream analysis of label distribution properties and their relationship to model performance.

4. Feature Generation

4.1 Technical Indicator Library (TA-Lib Based)

The primary feature generation module, `generate_all_indicators()` in `features/all_indicators.py`, wraps the TA-Lib library to compute 147 indicators across seven categories. Overlap studies include Bollinger Bands (20-period), DEMA, EMA at periods 10/20/50/200, Hilbert Transform Trendline, KAMA, simple and weighted moving averages, MAMA/FAMA, Parabolic SAR with extension, T3, TEMA, and TRIMA. Momentum indicators span ADX (14), ADXR, APO, AROON oscillator and components (14), BOP, CCI (14), CMO (14), DX (14), MACD with signal and histogram, MFI (14), MINUS_DI/PLUS_DI/DM (14), MOM (10), PPO, ROC and variants (10), RSI (14), Stochastic with fast and full variants (default periods), StochRSI K/D (14), TRIX (30), Ultimate Oscillator, and Williams %R (14).

Volume indicators include the Accumulation/Distribution Line (AD), A/D Oscillator (ADOSC), and On-Balance Volume (OBV). Volatility indicators cover Average True Range (ATR-14), Normalized ATR (NATR-14), and True Range (TRANGE). Price transforms provide `AVGPRICE`, `MEDPRICE`, `TYPPRICE`, and `WCLPRICE` as candlestick context features. Cycle indicators from the Hilbert Transform suite include `HT_DCPERIOD`, `HT_DCPHASE`, `HT_PHASOR` (in-phase and quadrature components), `HT_SINE` with lead sine, and `HT_TRENDMODE`. These cycle features capture dominant cycle periods and phase relationships that are invisible to standard trend-following indicators.

4.2 Statistical and Derived Features

Beyond TA-Lib indicators, the framework computes several derived statistical features. Rolling statistics (14-period) include `rolling_mean`, `rolling_std`, and `z_score` for mean-reversion signal generation. A Volume-Weighted Average Price (VWAP) feature captures the cumulative volume-weighted midpoint, useful as a reference level for institutional positioning. Hull Moving Average (HMA) is computed as $WMA(2 * WMA(close, n/2) - WMA(close, n), \sqrt{n})$ using a 10-period base. Keltner

Channel upper and lower bands, Donchian Channel 20-period high/low, Elder Force Index (EMA-13 of price-change times volume), and a basic Supertrend variant (close - $ATR \times 2$) are also included.

4.3 Candlestick Pattern Features

The framework computes 60 TA-Lib candlestick pattern recognition outputs, including classical reversal patterns such as CDL3BLACKCROWS, CDL3INSIDE, CDLENGULFING, CDLHAMMER, CDLHARAMI, CDLMARUBOZU, CDLSHOOTINGSTAR, CDLMORNINGSTAR, CDLEVENINGSTAR, and CDLDOJI variants. Each pattern output is a signed integer—positive for bullish signals, negative for bearish signals, zero for no pattern detected—that can serve as binary or ordinal features after appropriate normalization.

4.4 Session and Calendar Features

The features/fx_calendar.py module provides session classification and intraday calendar features critical for conditioning predictions on market regime. The classify_session_regime() function assigns each bar to one of five session regimes: 'overlap' (London-NY overlap, highest liquidity), 'london', 'new_york', 'asia', or 'off_hours'. Session boundaries are defined in the RegimeLabelConfig dataclass with configurable parameters: London 07:00-16:00 local, New York 08:00-17:00 local, and Asia 19:00-02:00 EST. The resulting session_regime label is used both as a feature and as a stratification variable in regime-conditional evaluation.

Intraday cyclical features (hour_sin, hour_cos using sinusoidal encoding of the hour of day) capture periodic patterns in EURUSD volatility and directionality without imposing discontinuities at midnight. Additional binary session membership indicators (in_asian_session, in_london_session, in_newyork_session) provide sparse session indicators that the model can use for direct interaction effects.

4.5 Feature Engineering Transforms

The features/transforms.py module implements several higher-order feature transformations applied to the raw feature set. ATR-normalized features scale price-

derived features by the contemporaneous ATR to provide volatility-invariant inputs. Rolling z-scores standardize features over configurable lookback windows, removing long-term trending behavior from mean-reverting indicators. Rolling winsorized features clip extreme values to reduce the influence of outliers on gradient-based models. Rolling percentile rank features transform absolute feature values into their percentile rank within a rolling window, which is both robust to outliers and directly interpretable as a market state percentile. Lag features and interaction features (products and ratios of selected feature pairs) provide temporal context and nonlinear combinations for the XGBoost lag-window features.

5. Feature Importance and Selection

5.1 Multi-Method Importance Computation

Feature importance in this framework is computed through multiple complementary methods and then merged into a unified ranking score. This multi-method approach is motivated by the known limitations of any single importance measure: Mean Decrease in Impurity (MDI) from tree-based models overestimates the importance of high-cardinality and correlated features; permutation importance (MDA) is computationally expensive and high-variance; SHAP values provide directionally informative attributions but can assign high importance to features that are correlated with the true driver. By combining multiple methods, the merged ranking degrades gracefully when any single method produces biased estimates.

The base ranking is derived from a `composite_score` column in the prepared target's `feature_importance.csv` file, which aggregates random forest feature importance (`rf_importance`), mutual information (`mutual_information`), and point-biserial correlation (`association`). These are computed during the data preparation phase, which runs upstream of model training. Backend-specific ranking files—`feature_importance_merged_xgboost.csv`, `feature_importance_merged_tcn.csv`, and `feature_importance_merged_lstm.csv`—allow the feature selection to be tailored to each model family's inductive bias, reflecting the empirical observation that tree-based and sequence-based models exploit different aspects of the same feature set.

5.2 SHAP Attribution Integration

SHAP (SHapley Additive exPlanations) values are computed post-training on the out-of-fold prediction set and summarized into a SHAP ranking frame with nine columns: `mean_abs_shap_all` (mean absolute SHAP across all samples), `mean_signed_shap_all` (signed mean indicating direction), `mean_feature_value_all` (mean raw feature value for context), and corresponding columns for the positive-label subset (`mean_abs_shap_positive`, `mean_signed_shap_positive`, `mean_feature_value_positive`) and negative-label subset. The `mean_abs_shap_positive` column specifically captures

which features are most important for generating positive predictions, which is more actionable for practitioners than overall SHAP importance when positive labels represent a small fraction of the dataset.

5.3 Merged Ranking Formula

The `merge_feature_rankings()` function in `model_training/ote_training/feature_ranking.py` combines the base ranking and SHAP ranking into a merged_score using a weighted sum of rank-normalized components. Each component is converted to a unit score in [0, 1] via `_rank_to_unit_score()`, which maps feature ranks (1 = highest importance) to linearly spaced scores (1.0 for rank 1, 0.0 for the last rank).

$$\text{merged_score} = (w_base * \text{base_rank_score} + w_shap * \text{shap_rank_score} + w_shap_pos * \text{shap_positive_rank_score}) / \text{active_weight_sum}$$

with `base_weight=0.50`, `shap_weight=0.35`, and `shap_positive_weight=0.15` as defaults. The `active_weight_sum` normalization ensures that if SHAP values are unavailable (all zero), the base ranking receives full weight without numeric inflation. Features are sorted by `merged_score` descending, with `mean_abs_shap_all` and `composite_score` used as tiebreakers, followed by feature name alphabetically for deterministic reproducibility. A `merged_rank` integer column records the final ordinal position.

5.4 Attribution Gate Filtering

After merging, the `filter_features_by_attribution_gates()` function applies sequential attribution gates to select the final feature subset for training. The first gate applies a floor filter: features with `mean_abs_shap_all` below `floor_fraction=0.15` times the maximum attribution are removed. This eliminates near-zero-contribution features that would add noise to the model while providing negligible signal.

The second gate applies cumulative importance selection: features are sorted by `mean_abs_shap_all` descending, and the minimal set of top features whose cumulative importance sum reaches `cumulative_importance=0.90` of the total SHAP mass is retained. This ensures that the selected feature set explains at least 90% of the model's total SHAP

attribution mass while eliminating low-contribution features that add degrees of freedom without proportional benefit. The function returns both the filtered ranking frame and a diagnostic summary with `input_feature_count`, `after_floor_count`, `selected_feature_count`, and `selected_cumulative_importance_share`.

5.5 Leakage Detection and Feature Validation

A critical safeguard in the pipeline is the `blocked_for_leakage()` function, which scans feature names against a hardcoded set of 12 look-ahead contamination patterns: `'target_profit'`, `'future'`, `'lookahead'`, `'forward_'`, `'_forward'`, `'pnl'`, `'mfe_'`, `'_mfe'`, `'mae_'`, `'_mae'`, `'take_profit'`, `'tp_hit'`, `'sl_hit'`, `'exit_signal'`. Any feature whose name contains one of these patterns is unconditionally removed before model training, regardless of its importance score. This convention-based leakage detection acts as a last-line defense against accidental inclusion of future-dependent features that might survive the importance filtering step due to artificially high scores.

The feature pipeline addresses multicollinearity indirectly through the attribution gate's cumulative importance selection: when two features are highly correlated, they typically share SHAP mass, and SHAP's marginal attribution framework tends to split importance across correlated features rather than double-counting. However, for the XGBoost lag window features specifically, the `delta_feature_cap` parameter (default 16) limits the number of features for which first-difference lag features are computed, reducing the explicit temporal redundancy that would arise from including both raw and differenced versions of all features.

5.6 Feature Count and Selection per Model

The `OTETrainingConfig` dataclass controls feature selection geometry: `max_loaded_features=160` sets the hard cap on features loaded from the ranking file before hyperparameter-specific selection; `top_feature_min=24`, `top_feature_max=128`, and `top_feature_step=8` define the Optuna search space for the number of top features used in each trial. This means that each Optuna trial selects a different feature count from

{24, 32, 40, ..., 128}, and the optimal count is determined empirically by the combined objective function rather than fixed a priori.

Table 2. Feature importance methods, their data scope, and weights in the merged ranking.

Method	Scope	Column Used	Weight	Addresses
Random Forest MDI	Training set	rf_importance	Indirect via composite_score	Non-linear importance
Mutual Information	Training set	mutual_information	Indirect via composite_score	Non-linear dependence
SHAP Mean Absolute	OOF predictions	mean_abs_shap_all	0.35	Marginal attribution
SHAP Positive Class	OOF predictions (pos)	mean_abs_shap_positive	0.15	Precision-relevant features
Cumulative Gate	Post-merge	merged_score	—	Dimensionality control

6. Model Training and Hyperparameter Analysis

6.1 Training Architecture Overview

All OTE models are trained through the unified `ote_xgboost_pipeline.py` entry point, which despite its name supports three backends selectable via the `--backend` flag: 'xgboost', 'torch' with `model_type='tcn'`, and 'torch' with `model_type='lstm'`. The trainer operates on prepared target directories containing chronological train/val/test splits, a `features.json` allowlist, a feature ranking CSV, and a `report.json` with dataset metadata. The `OTETrainingConfig` dataclass centralizes all training hyperparameter ranges and architecture defaults.

6.2 XGBoost with Sparse Geometric Lag Windows

The XGBoost backend uses a sparse lag representation to inject temporal context into what would otherwise be a single-bar feature vector. The `make_lag_steps()` function generates a geometrically spaced set of lag offsets: given `window_size` bars and `lag_count` lag indices, the steps are computed by `np.geomspace(1, window_size-1, num=lag_count-1)`, rounded to integers, with `lag_step=0` (current bar) and `lag_step=window_size-1` (oldest bar) always included. This geometric spacing allocates more resolution to recent time steps—which are more predictive of short-term entries—while still providing long-range context.

The `build_sparse_lag_view()` function constructs the windowed feature matrix by horizontally concatenating the feature vectors at each lag offset. For a window of `w` features and `L` lag steps, the output matrix is `[rows x (w*L)]`. Additional first-difference features are appended for the top `delta_feature_cap=16` ranked features, computing `(current_value - lagged_value)` at each lag step. These delta features capture the velocity of indicator changes without requiring the model to learn the subtraction itself.

XGBoost hyperparameters are tuned via Optuna TPE sampling with MedianPruner over `n_trials=20` trials. The Optuna search space spans: number of top features (24-128 in steps of 8), window size (8-40), lag count (4-8), XGBoost `n_estimators` (100-800), `learning_rate` (0.005-0.15 log-uniform), `max_depth` (3-8), `subsample` (0.5-1.0),

colsample_bytree (0.4-1.0), min_child_weight (1-20), gamma (0-5), reg_lambda (0.1-10), and scale_pos_weight (class imbalance weight, 1-50). Each trial runs purged walk-forward CV and the Optuna objective is a weighted combination of CV average precision and CV event F0.5 score.

6.3 Temporal Convolutional Network (TCN) Architecture

The TCNClassifier in torch_models.py implements a dilated causal residual network. Each TCNResidualBlock contains two CausalConv1d layers with left-padding to maintain sequence length, GELU activations, dropout regularization, and a residual connection with a pointwise 1x1 convolution to match channel dimensions when input and hidden sizes differ. The dilation pattern follows powers of two: dilations = $[2^0, 2^1, \dots, 2^{(\text{num_layers}-1)}]$, which gives exponentially growing receptive fields. With kernel_size=3 and num_layers=4, the theoretical receptive field covers $2 \cdot (3-1) \cdot (2^0 + 2^1 + 2^2 + 2^3) = 60$ timesteps, sufficient for the default window_size=24 bar context window.

The model's forward pass transposes the input from [batch, time, features] to [batch, features, time] for the Conv1d operations, processes through all dilated residual blocks, and reads out the features at the final (most recent) timestep $x[:, :, -1]$ before the classification head. This design aligns with the current-bar prediction objective: the model's output represents the probability that the current bar is a valid entry, conditioning on all prior bars in the window. A final dropout and Linear(hidden_size, 1) head projects to a scalar logit.

An important implementation detail is the resolve_amp_usage() function, which disables automatic mixed precision for TCN training even when CUDA is available. This decision was made after observing non-finite logits during full-data refits with AMP enabled on TCN models, a known issue with deep dilated convolutional architectures and float16 gradient accumulators. LSTM models retain AMP support.

6.4 LSTM Architecture

The `LSTMClassifier` implements a standard stacked LSTM with `batch_first=True`, `num_layers`-aware inter-layer dropout (`lstm_dropout = dropout if num_layers > 1 else 0.0` to avoid a PyTorch warning), and a classification head that reads from the final hidden state `hidden_state[-1]`. This last-hidden-state readout is appropriate for the entry timing task: the LSTM accumulates all temporal context through its recurrent state, and the final state represents the model's summary of the entire sequence up to the current bar.

6.5 Weighted Focal Loss and Hard Negative Mining

The `WeightedFocalLoss` class in `torch_trainer.py` implements the focal loss function (Lin et al., 2017) adapted for binary classification with per-sample weighting. The loss for a single sample is:

$$L_{focal} = -\alpha_t * (1 - p_t)^\gamma * BCE(\text{logit}, \text{target})$$

where `p_t` is the model's predicted probability for the true class, `alpha_t` is the class-specific weighting (`alpha` for positive class, `1-alpha` for negative class), and `gamma` is the focusing parameter that down-weights easy examples. Sample weights from the label quality and uniqueness computation are applied multiplicatively to the per-sample focal loss before averaging. The `focal_alpha` hyperparameter is searched in `[0.70, 0.95]` and `focal_gamma` in `[1.25, 3.75]`, tuned by Optuna for each target and fold.

Hard negative mining is implemented by computing the `hard_negative_radius`—the number of bars around each confirmed positive event that are explicitly excluded from random negative sampling—and `hard_negative_multiplier`, which oversamples the negatives immediately outside this radius. This directs gradient updates toward the ambiguous boundary region between genuine entry opportunities and near-miss configurations, improving the model's discrimination at operationally relevant probability thresholds.

6.6 Training Phase Schedule

The `build_phase_schedule()` function in `torch_trainer.py` implements a three-phase training curriculum for PyTorch models. In automatic mode (no explicit phase

specification), the schedule divides `total_epochs` into three phases: warmup (1/5 of total epochs, `train_fraction=0.35`), main (3/5 epochs, `train_fraction=1.0`), and fine (1/5 epochs, `train_fraction=1.0`). The warmup phase uses only 35% of the training data, allowing the model to stabilize on a representative but small subset before exposure to the full class imbalance. The fine phase uses a reduced learning rate (`fine_lr_scale=0.35` default) to enable careful convergence after the main training has established the primary decision boundaries.

Explicit phase scheduling is also supported for fine-grained control, as documented in `notes.txt`: 'rerun with 48 epochs, window 28, focal gamma 3.70, hard-negative multiplier 1.37'—reflecting empirical tuning insights from the `long_breakout_tcn` repair cycle where the model was still improving at epoch end. The explicit schedule format (`torch_warmup_epochs + torch_main_epochs + torch_fine_epochs + torch_tail_epochs = total_epochs`) must sum exactly to `total_epochs`, enforced by runtime validation.

6.7 Purged Walk-Forward Cross-Validation

The cross-validation design is the single most important methodological choice for preventing overfitting to in-sample autocorrelation structure. The framework uses an expanding-window walk-forward scheme defined by four geometry parameters: `cv_initial_train_rows=250,000` (the minimum training set for fold 1), `cv_val_rows=100,000` (validation window size), `cv_step_rows=100,000` (training set growth per fold), and `cv_max_train_rows` (optional cap). With these defaults and approximately 2.5 million bars, the scheme produces 8 or more folds for full-dataset training runs.

Between each training fold and its corresponding validation fold, a `purge_buffer_rows` gap (default 12) removes bars at the chronological boundary to ensure that any observation whose label horizon extends into the validation period is excluded from training. This implements the purging concept from Lopez de Prado (2018, Ch. 7.1) at the fold level, complementing the within-sample exclusion zones implemented at the label level.

Scaling is performed inside each fold using RobustScaler fit exclusively on the training partition of that fold, with `scale_quantile_low=5.0`, `scale_quantile_high=95.0`, and `scale_clip=8.0` to prevent extreme test-set values from corrupting predictions. The scaler is never fit on validation or test data.

6.8 Probability Calibration

Raw model probabilities from XGBoost and PyTorch models are typically uncalibrated: their predicted probabilities do not match empirical positive rates. This miscalibration is particularly severe for the focal loss objective, which deliberately depresses high-confidence probabilities to focus gradient updates on uncertain examples. The framework supports two calibration methods selectable via `calibration_method`: 'platt' (logistic regression fit to out-of-fold predictions) and 'isotonic' (isotonic regression, a non-parametric monotone transformation). Calibrators are fit on the combined train+val out-of-fold predictions before final test evaluation, and the calibrated probabilities are exported alongside raw probabilities.

Table 3. Model family performance on the training leaderboard (42 artifacts) vs. best post-training Sharpe ratio.

Model Family	Backend	CV AP Range	CV F0.5 Range	Best CV AP	Best Post-Training Sharpe
Breakout TCN	torch/tcn	0.950-0.969	0.875-0.943	0.969	0.81
Breakout XGB	xgboost	0.923-0.933	0.906-0.925	0.933	neg.
OTE/Union TCN	torch/tcn	0.625-0.862	0.773-0.885	0.862	2.40
Reversal TCN	torch/tcn	0.580-0.663	0.692-0.775	0.663	3.28
Reversal XGB	xgboost	0.571-0.694	0.795-0.797	0.694	3.61
LSTM	torch/lstm	0.277-0.519	0.434-0.691	0.519	n/a

7. Model Testing, Regimes, and Backtesting

7.1 Pairwise Regime Detection

Regime detection in this framework is implemented as a deterministic, rule-based labeling system rather than a statistical model such as a Hidden Markov Model or Gaussian Mixture Model. This design choice reflects several practical considerations. Rule-based regimes are fully reproducible across time and data updates without retraining; they are interpretable by practitioners who need to understand what market condition is driving a policy decision; and they avoid the regime-switching instability that plagues unsupervised regime models when applied to new market periods.

The `composite_regime` is constructed as the cross-product of two independent regime axes: `trend_regime` and `vol_regime`. The pairwise interaction of these two dimensions defines 15 distinct regime cells that capture the joint state of market direction and volatility level. The `composite_regime` column is computed as `trend_regime + '_' + vol_regime`, yielding labels such as `'strong_up_high'`, `'ranging_low'`, `'weak_down_medium'`, etc. This is implemented in the `label_regimes()` function in `model_testing/ote_regime_labeler.py`, which is called on the joined prediction frame as a post-training step.

The `trend_regime` axis uses ADX and EMA alignment as its primary signals, implemented in `_resolve_trend_regime()`. When the `adx_14` column is available, the classification proceeds in order of precedence: ADX below 20 (`adx_ranging_threshold`) -> `'ranging'`; ADX between 20 and 30 (`adx_strong_threshold`) with close above EMA-50 -> `'weak_up'`; ADX between 20 and 30 with close below EMA-50 -> `'weak_down'`; ADX above 30 with `ema_alignment` above 0.7 (`ema_alignment_strong_threshold`) -> `'strong_up'`; ADX above 30 with `ema_alignment` below -0.7 -> `'strong_down'`. The `ema_alignment` feature itself is a normalized spread between a fast and slow EMA pair, scaled by ATR.

The `vol_regime` axis is computed from the ATR percentile rank over a 504-bar rolling window (approximately 42 hours at 5-minute resolution). Bars where the ATR percentile

rank falls below the 25th percentile are classified as 'low' volatility; above the 75th percentile as 'high' volatility; all others as 'medium'. This percentile-based classification automatically adapts to long-run changes in EURUSD absolute volatility levels, ensuring that 'high volatility' means the same relative thing in 2012 and 2024.

Why pairwise rather than single-feature regimes? A single ADX threshold classifies all trending periods together regardless of whether they are high-volatility breakout trends or low-volatility grinding trends—two conditions that have very different risk profiles for entry models. Conversely, a volatility-only regime ignores whether a high-volatility episode is trending or ranging, which determines whether momentum-following or mean-reverting entry strategies are appropriate. The pairwise interaction captures what single-variable classification misses: the joint configuration of direction strength and volatility amplitude that determines the economic character of the trading environment.

Two additional regime dimensions complement the composite: `session_regime` (overlap, london, new_york, asia, off_hours) from the FX calendar module, and `stress_regime` (normal, elevated, high) derived from the `range_shock_20` feature—a 20-bar z-score of the bar range relative to its rolling mean. The `stress_regime` identifies abnormal volatility spikes such as economic data releases or geopolitical events that fall outside the range of normal intraday variation. The `DEFAULT_SLICE_FAMILIES` tuple in `ote_regime_slices.py`—('composite_regime', 'trend_regime', 'vol_regime', 'session_regime', 'stress_regime', 'year')—specifies all dimensions evaluated in the regime slice report.

7.2 Regime-Conditional Evaluation

The `build_regime_slice_report()` function in `ote_regime_slices.py` generates per-regime evaluation tables by iterating over each regime family and each unique value within that family. For each regime bucket, it computes: `rows_in_bucket`, `n_positive` (event count), `n_negative`, AP (average precision with bootstrap 95% CI), `event_precision`, `event_recall`, `event_f05` (F-beta with $\beta=0.5$, precision-weighted), and both current-threshold and optimal-threshold metrics.

The event-based metrics differ from standard point-by-point precision/recall by operating on zones of consecutive positive bars rather than individual bar predictions. The `group_positive_zones_by_position()` function groups consecutive positive bars into event zones by their `source_row_idx` positions, using a maximum gap criterion. A prediction is considered a true positive if it falls within `event_tolerance_bars=2` bars of any true event zone. This tolerance reflects the economic reality that being one or two bars early or late in predicting a swing entry is not meaningfully different from being exactly on the event bar.

Bootstrap confidence intervals for average precision (200 iterations, 95% CI) are computed for each regime bucket using a stratified resampling scheme that preserves the positive-to-negative ratio. These intervals are used in `build_regime_winner_table()` to determine whether one model's advantage over another in a given regime is statistically significant: the `confidence_gap = winner.ap_ci_lower - runner_up.ap_ci_upper` is reported as a proxy for the significance of the win.

Regime performance analysis across the champion and challenger model set consistently reveals that model breakdown conditions align with regime boundaries: models that perform well in 'strong_up_medium' often degrade in 'ranging_low', and models tuned for 'ranging_medium' may generate excessive false positives in 'weak_down_high' conditions. The composite regime evaluation thus serves both as a diagnostic tool for understanding model failure modes and as a calibration surface for regime-dependent threshold selection.

7.3 Threshold Calibration

Prediction threshold selection is implemented as a search over the `DEFAULT_THRESHOLD_GRID = np.arange(0.40, 0.901, 0.05)`, yielding 11 candidate thresholds from 0.40 to 0.90. The `search_thresholds_by_composite_regime()` function in `ote_threshold_policy.py` performs this search independently for each `composite_regime` bucket, enabling the operating threshold to adapt to the model's calibration characteristics within each market regime.

For each threshold candidate t , the event-based metrics are computed using `event_metrics_by_position()`: a prediction is emitted when the model probability exceeds t , subject to `event_cooldownBars=4` bar minimum spacing between consecutive predictions. Cooldown enforcement prevents the model from generating a burst of predictions around a single structural event, which would count as multiple predicted events but correspond to only one true trade opportunity.

The threshold selection objective combines event F-beta score and precision via a weighted scoring formula defined by `threshold_event_fbeta_weight=0.55` and `threshold_event_precision_weight=0.45`, plus a turnover penalty parameterized by `threshold_turnover_penalty_weight=0.40` and `threshold_turnover_target_ratio=0.85`. The turnover penalty discourages thresholds that generate significantly more predicted events than the optimal coverage rate would suggest, operationalizing the preference for fewer but higher-quality trades.

A leave-one-year-out validation is applied when multiple years of data are available: the threshold search is performed with each calendar year held out in turn, and the threshold that maximizes the minimum objective across holdout years is selected. This step is used `used_leave_one_year_out=True` in the result dictionary and provides additional robustness against overfitting to idiosyncratic features of a single calendar year.

Default threshold hints per regime direction are hard-coded in `LONG_THRESHOLD_HINTS` and `SHORT_THRESHOLD_HINTS` as a prior: for example, `'ranging_low'` defaults to 0.70 for both long and short, while `'strong_up_low'` uses 0.55 for long entries but 0.80 for short entries (reflecting the asymmetric risk of counter-trend short entries in strong uptrends). These hints serve as fallback values when a regime bucket has insufficient data (`< min_positive_events=50` true events) for statistically reliable threshold search.

7.4 Abstain Policy Layer

Beyond threshold filtering, the `HardAbstainConfig` in `ote_abstain_policy.py` defines explicit hard rules for abstaining from signal emission regardless of model probability.

The `apply_abstain_policy()` function processes signal candidates sequentially through a hierarchy of abstain conditions, recording the first matching reason in `policy_abstain_reason`. The abstain conditions in order of precedence are:

1. High stress (`abstain_high_stress=True`): any bar where `stress_regime='high'` is blocked. This prevents entry during abnormal volatility spikes where spread costs and slippage are likely to exceed the expected move.
2. Off-hours (`abstain_off_hours=True`): signals during `session_regime='off_hours'` are blocked due to extremely wide spreads (3.0 pips default in `DEFAULT_SESSION_SPREAD_PIPS`).
3. Explicit session/composite/pair filters: configurable via `abstain_session_regimes`, `abstain_composite_regimes`, and `abstain_composite_session_pairs` tuples for deployment-specific regime pruning.
4. Expected-move filter: when `expected_move_by_regime` is provided, signals are blocked if the estimated expected move ($\text{probability} \times \text{regime_expected_move}$) falls below `minimum_expected_move_to_spread=2.0` times the session-specific spread. This implements a cost-of-carry filter that prevents the model from generating signals when the gross expected return cannot exceed transaction costs with reasonable probability.
5. Cooldown: after any accepted signal, `cooldownBars=4` subsequent bars are blocked unconditionally, preventing burst trading around single structural events.

7.5 Walk-Forward Policy Backtesting

The walk-forward backtest is the framework's primary deployment-readiness evaluation tool. The `run_walk_forward_backtest()` function in `ote_policy_backtest.py` orchestrates the full simulation across folds defined by `build_walk_forward_folds()`. The fold generation uses a rolling quarterly scheme: starting from the first test window that has at least `min_train_years=2` years of training data preceding it, test windows of `test_window_months=3` months are stepped forward by `rolling_step_months=3` months until the end of the dataset. A `purge_gap_bars=40` bar gap is enforced between the last training position and the first test position, larger than the intra-fold purge buffer to account for the possibility that threshold-search optimization on the training fold implicitly sees some structure that extends into the test fold.

Within each fold, the training partition is used to search thresholds via `search_thresholds_by_composite_regime()`, producing a per-regime policy table that maps composite regimes to calibrated thresholds. This policy is then applied to the test partition via `apply_threshold_policy()`, which looks up each bar's composite regime and applies the regime-specific threshold. The abstain policy is applied on top of the threshold policy to produce the final `policy_emit_signal` column.

Trade-level outcomes are computed by `attach_forward_trade_outcomes()` using a simplified simulation: at each emitted signal, the model enters at the next bar's close and holds for `label_max_holding_bars=120` bars or until the first Triple Barrier hit. Gross P&L in pips is computed as $(\text{exit_price} - \text{entry_price}) / \text{pip_size}$ for longs and the inverse for shorts. Net P&L deducts session-specific spread costs from `DEFAULT_SESSION_SPREAD_PIPS` (overlap: 1.0, london: 1.5, new_york: 1.5, asia: 2.5, off_hours: 3.0), plus optional slippage modeled as `slippage_spread_multiplier=0.25` times the `approx_spread` column, and any fixed commission.

The backtest acceptance gates implemented in `WalkForwardBacktestConfig` enforce six quantitative criteria that a model must pass before being considered deployment-ready: (1) minimum `min_trades_per_week=3.0` average signals per week; (2) minimum Walk-Forward Efficiency `minimum_wfe=0.50`; (3) maximum single-trade concentration `max_single_trade_share=0.10` (no single trade contributes more than 10% of total gross PnL); (4) minimum profitable quarter share `minimum_profitable_quarter_share=0.60`; (5) minimum positive composite regime share `minimum_positive_composite_share=0.60`; and (6) overall positive net PnL. Models that pass all six gates are considered 'full-gate' champions; models passing 5/6 are considered candidates with a specific identified weakness for targeted improvement.

7.6 Walk-Forward Efficiency and Performance Metrics

Walk-Forward Efficiency (WFE) is defined as the ratio of out-of-sample test P&L to in-sample training P&L, computed at the fold level and averaged across all folds. A WFE above 0.50 indicates that the model retains at least 50% of its training-period edge when

applied to unseen data, a benchmark that guards against severe overfitting. The `summarize_walk_forward_efficiency()` function aggregates fold-level WFE statistics and flags whether the minimum threshold is met.

Additional performance metrics include: monthly Sharpe ratio (annualized net PnL per month divided by monthly PnL standard deviation); profit factor (total winning trade PnL / total losing trade PnL); maximum drawdown in pips from equity curve; and a breakout by calendar year, calendar quarter, session, composite regime, and confidence quintile via the `build_group_breakdown()` function. The confidence quintile breakdown reveals whether high-confidence model outputs (quintile 5) consistently outperform low-confidence outputs (quintile 1), validating the calibration of the probability outputs.

7.7 Results Across Regimes and Models

The post-training leaderboard compiled from 63 saved backtest runs across 19 unique model identifiers reveals several important findings. First, the training leaderboard rank has minimal predictive value for post-training performance: the top training-CV model (`long_breakout_tcn_challenger`, CV AP=0.969) has no profitable post-training backtest, while the top post-training model (`short_reversal_xgb_v2_20260525`, monthly Sharpe=3.61) ranks only 19th by training CV AP (0.694).

Second, the reversal family shows consistently superior walk-forward performance compared to breakout and OTE families. The top six post-training positions are all reversal models, all passing all six acceptance gates. The `short_reversal_xgb_v2` achieves monthly Sharpe 3.61, profit factor 2.74, WFE 0.73, and 100% profitable quarter share. The `long_reversal_tcn_v3` achieves annualized net pips 3,558 with profit factor 2.58 and WFE 0.796. By contrast, breakout models produce high training CV scores but convert poorly to profitable policies: `short_breakout_tcn_champion` shows monthly Sharpe -0.95 and only 2/6 gates passed in its best saved run.

Third, OTE/union TCN models occupy the middle ground: `long_ote_tcn_v2_candidate` achieves the highest single-model annualized net pips (3,955) but passes only 5/6 gates due to concentration risk, while `short_ote_meta_tcn_champion` (Sharpe 2.70, 6/6 gates)

and long_ote_union_tcn_candidate (Sharpe 2.40, 6/6 gates) demonstrate robust performance with full gate compliance.

Table 4. Top 10 models by post-training monthly Sharpe ratio with key acceptance criteria.

Model ID	Direction	Backend	Gates	Monthly Sharpe	Ann. Net Pips	Profit Factor	WFE
short_reversal_xgb_v2	short	xgboost	6/6	3.61	2729	2.74	0.73
long_reversal_tcn_v2	long	tcn	6/6	3.36	2736	2.80	0.82
long_reversal_tcn_v3	long	tcn	6/6	3.28	3558	2.58	0.80
short_reversal_xgb_v1	short	xgboost	6/6	2.91	1004	2.97	1.04
short_reversal_tcn_champion	short	tcn	6/6	2.78	2732	2.77	2.00
long_reversal_tcn_champion	long	tcn	6/6	2.76	2464	3.07	0.80
short_ote_meta_tcn	short	tcn	6/6	2.70	1737	1.91	0.99
long_ote_union_tcn	long	tcn	6/6	2.40	2831	1.72	1.22
short_ote_union_tcn	short	tcn	6/6	1.87	1241	1.73	1.10
long_ote_tcn_v2	long	tcn	5/6	2.36	3955	1.96	1.98

7.8 Dissociation Between Training and Post-Training Rankings

The observed dissociation between CV AP and post-training Sharpe deserves careful analysis because it has direct implications for model selection methodology. The breakout family's extremely high CV AP (0.950-0.969) arises because breakout labels are relatively clean and structurally distinctive: a compression-to-expansion bar has a distinctive feature profile that a gradient boosted model can learn reliably within the training folds. However, this same structural distinctiveness makes breakout models fragile in deployment: they learn narrow feature configurations that may not generalize to slightly different market conditions, and the label's economic content—the profitability of entering on a breakout bar—degrades rapidly when spreads are modeled.

Reversal models, by contrast, learn from noisier structural events with lower average label quality, producing lower CV AP scores. But the economic character of reversal trades is more robust: a genuine structural swing provides a natural stop level (beyond the swing extreme) and a favorable risk-reward profile that persists across different volatility and trend contexts. This explains why reversal models with CV AP in the 0.55-0.70 range consistently outperform breakout models with CV AP above 0.90 in post-cost walk-forward simulations.

This dissociation motivates the framework's key operational principle documented in `MODEL_LEADERBOARDS_20260527.md`: 'Use the post-training leaderboard, not the raw training leaderboard, as the primary promotion surface.' The recommendation to raise the selection bar by requiring at minimum three CV folds and a passing post-training proxy constraint before promoting a model to the candidate tier operationalizes this principle.

8. Discussion

8.1 What Worked: Structural ATR and Event-Driven Labels

The most impactful methodological contribution of this framework is the structural ATR design. By computing swing detection thresholds from the 1-hour ATR and mapping them to the 5-minute bar grid, the labeling system avoids the consolidation clustering problem that afflicts naive 5-minute ATR based labelers. The resulting label distribution is well-calibrated: a `positive_rate_long` of 1-2% matches the expected frequency of genuine institutional swing entries in EURUSD, and the inter-swing coefficient of variation ($CV < 3.0$) diagnostic in `run_diagnostics()` confirms that swings are well-distributed rather than concentrated in volatile periods.

The trend-scan statistical validation step proved to be a powerful noise filter. By requiring that the t-statistic of a linear trend fit over multiple forward windows exceed `trend_scan_min_abs_t=2.0`, the labeler eliminates structural swing points that were not followed by directional momentum—exactly the 'false' swing labels that would degrade model quality. The improvement in `label_quality_mean` observed after enabling `trend_scan_enforce=True` represents a meaningful increase in the economic content of each positive label.

The regime-conditional threshold search was another effective addition. Allowing the operating threshold to differ by composite regime captures the empirical reality that model confidence scores have different calibration characteristics in trending versus ranging markets. The `LONG_THRESHOLD_HINTS` reveal the expected pattern: lower thresholds (0.55-0.60) in favorable regimes (`strong_up`) and higher thresholds (0.70-0.75) in adverse or ambiguous regimes, matching the practitioner intuition that one should be more selective when the macro environment is not supportive.

8.2 What Did Not Work: Breakout Models and High CV AP

The breakout labeling family, despite producing the cleanest labels and highest-CV-AP models in training, failed to convert to profitable post-cost policies. The root cause appears to be the combination of tight holding windows (`tb_maxBars=24`) and the

requirement for the price to already have broken the range before the label is assigned. This means that the model is being asked to predict an event that has already largely happened, and the label-consistent entry—on the breakout bar itself—is often at an unfavorable price relative to the subsequent expected move. The `notes.txt` entry 'short_breakout_tcn_champion: treat this as an upstream pipeline issue, not a threshold issue... review event definition, holding horizon, TP/SL mapping, cooldown, and cost sensitivity' correctly diagnoses the problem as a training target mismatch rather than a model quality issue.

The LSTM family consistently underperformed TCN and XGBoost models across all evaluation dimensions. `ote_full_lstm_v2` ranks 41st out of 42 training artifacts with CV AP=0.277 and CV F0.5=0.434. This underperformance likely reflects the LSTM's sensitivity to the class imbalance problem: without the focal loss focusing mechanism, the LSTM tends to learn to predict the majority class with high confidence, and the cross-entropy gradient is dominated by the overwhelming number of negative examples. While the focal loss is applied, the LSTM's recurrent gradient dynamics appear less compatible with the hard-negative mining regime than the TCN's parallel convolutional structure.

8.3 Surprising Findings

The WFE values above 1.0 for several models (`short_reversal_xgb_v1`: WFE=1.04, `long_ote_tcn_v2_candidate`: WFE=1.98) are initially surprising—they imply that the model performs better out-of-sample than in-sample. This phenomenon can arise when the training period contains higher-than-average noise or when the model was tuned on a more volatile period and then deployed in a calmer one. It could also reflect genuine positive transfer: a model trained on 2010-2022 data may benefit from EURUSD structural patterns that strengthen in the 2022-2026 period. Regardless of cause, WFE > 1.0 should be treated cautiously—it may indicate that the model has been underfitting the training data rather than that it has extraordinary generalization ability.

The near-perfect CV ROC-AUC scores (0.946-0.999 across all models) initially appear suspicious but are consistent with the highly imbalanced binary classification task: the

vast majority of bars are clearly negative, and AUC effectively measures rank ordering rather than calibration, making near-perfect AUC trivially achievable when the positive class is well-separated from the bulk of the negative class. Average precision, which focuses on the precision-recall tradeoff in the positive class, provides a much more discriminating performance metric and should be the primary training evaluation criterion.

8.4 Practical Implications for Financial ML Community

Several methodological conclusions from this framework have broad applicability to the financial ML community. First, the structural ATR normalization approach—computing thresholds from a higher timeframe and mapping them causally to the prediction timeframe—is a general solution to the consolidation clustering problem and should be considered for any event-driven labeling system in intrabar FX and equity data.

Second, the mandatory post-training acceptance gate framework provides a template for responsible model deployment. Requiring models to pass six quantitative criteria—including minimum trade frequency, WFE, concentration limits, and profitable quarter share—before promotion creates a principled barrier against overfitting to specific market periods. This is more robust than relying on CV scores alone and matches the standard of care required in institutional quantitative strategy development.

Third, the dissociation between training AP and post-training Sharpe ratio in this dataset should give pause to any practitioner who uses in-sample metrics as the primary model selection criterion. The magnitude of the dissociation—the best-CV model is post-cost unprofitable while mid-table CV models have Sharpe ratios above 3.0—suggests that the economic content of the training labels is a better predictor of post-training performance than the model's ability to fit those labels.

8.5 Limitations

Several important limitations constrain the generalizability of this framework's results. Transaction cost modeling uses simplified session-spread proxies derived from typical retail FX broker spreads; institutional execution with market impact, partial fills, and

dynamic spreads during news events would produce different cost structures. The walk-forward simulation assumes immediate fill at the close price of the signal bar, which overstates execution quality in practice, particularly for the reversal models whose entries occur near structural extremes where liquidity is typically thinner.

The labeling engines use forward-looking information (Triple Barrier outcomes, trend-scan t-statistics, entry followthrough scores) to generate training labels, which is appropriate for supervised learning but means that the model is trained on labels that can only be computed in hindsight. The assumption that the model can learn a proxy for this forward information from contemporaneous features is the fundamental bet of the supervised approach to trading, and it remains an empirical question whether this learning generalizes beyond the training distribution.

Finally, the framework currently focuses exclusively on EURUSD five-minute data. Generalization to other currency pairs, different timeframes, or equity/futures instruments would require re-validation of label parameter choices, particularly the structural ATR thresholds and Triple Barrier holding windows, which are calibrated to EURUSD's specific volatility characteristics.

9. Conclusion

This paper presents a comprehensive machine learning framework for EURUSD Optimal Trade Entry prediction that addresses the key methodological gaps between academic financial ML and practical trading system design. The framework's primary contributions span four areas: (1) a three-family adaptive labeling system using structural ATR thresholds, Triple Barrier validation, trend-scan statistical confirmation, and Lopez de Prado-style purging and sample uniqueness weighting; (2) a multi-method feature importance pipeline combining MDI, mutual information, SHAP all-samples, and SHAP positive-class attribution with sequential attribution gates; (3) purged walk-forward cross-validation with geometric lag windows for XGBoost and causal dilated TCN architectures with focal loss and hard negative mining for sequence models; and (4) a regime-conditional post-training evaluation framework using 15-cell composite regimes, event-based precision/recall metrics, and a six-gate walk-forward acceptance protocol.

The key empirical finding—that training-phase CV average precision is a poor predictor of post-training deployment performance, with the best-CV models (breakout family, AP 0.95-0.97) being post-cost unprofitable while mid-table models (reversal family, AP 0.55-0.70) achieving monthly Sharpe ratios of 2.76-3.61—has immediate practical implications for financial ML practitioners. It argues for a mandatory post-training evaluation stage that simulates realistic deployment conditions before any model is promoted to production consideration.

Several productive directions for future work emerge from this analysis. Adaptive regime detection using statistical change-point models or self-supervised regime embeddings could replace or supplement the rule-based composite regime, potentially identifying regime transitions earlier and with greater nuance. Reinforcement learning from trade-level outcomes could replace the supervised label framework entirely, allowing the model to optimize directly for post-cost risk-adjusted returns rather than label precision. Multi-asset ensemble approaches combining EURUSD signals with correlated pairs (GBPUSD, AUDUSD, DXY) could improve regime identification and reduce idiosyncratic noise. Finally, transfer learning across currency pairs—pre-training on high-data-volume pairs

and fine-tuning on lower-volume instruments—could extend the framework's reach without the full data requirements of from-scratch training.

Appendix A: Key Implementation Details

A.1 ReversalParams Default Configuration

The following configuration defines the default ReversalParams used in production labeling:

```
ReversalParams(  
    atr_period=14,                # Wilder's ATR period  
    atr_smoothing='ema',         # Wilder's EMA smoothing  
    structural_atr_tf='1hr',     # Use 1-hour ATR for thresholds  
    cusum_atr_mult=0.5,         # CUSUM threshold = 0.5 x structural ATR  
    confirm_atr_mult=0.8,        # Retrace for swing confirmation  
    min_swing_atr=1.35,          # Min swing amplitude in ATR units  
    min_swing_distance_atr=0.75, # Min price gap between swings  
    minBars_between_swings=18,   # Min 90 min at 5m between swings  
    tb_profit_atr=1.0,           # TP = entry + 1 x ATR  
    tb_stop_atr=2.0,             # SL = entry - 2 x ATR  
    tb_max_bars=120,             # 10-hour max holding period  
    trend_scan_windows=(12,24,48,96), # 1h to 8h forward windows  
    trend_scan_min_abs_t=2.0,    # Min t-stat for directional confirmation  
    min_label_quality=0.35,      # Minimum quality score for inclusion  
    exclusion_pre_bars=10,       # Purge zone before confirmation  
    warmup_bars=50               # Bars excluded from label statistics  
)
```

A.2 OTETrainingConfig Default Hyperparameter Ranges

The Optuna search space for OTE model training is defined by:

```
OTETrainingConfig(  
    n_trials=20,                  # Optuna trials  
    cv_initial_train_rows=250_000, # ~17 months at 5m  
    cv_val_rows=100_000,          # ~7 months validation window  
    cv_step_rows=100_000,         # Expanding step size  
    cv_min_folds=2,               # Minimum acceptable folds  
    max_loaded_features=160,      # Feature loading cap  
    top_feature_min=24,           # Search range for feature count  
    top_feature_max=128,  
    top_feature_step=8,  
    window_min=8,                 # Lag window size range  
    window_max=40,  
    lag_count_min=4,              # Geometric lag steps range  
    lag_count_max=8,  
    focal_alpha_min=0.70,         # Focal loss alpha range  
    focal_alpha_max=0.95,  
    focal_gamma_min=1.25,         # Focal loss gamma range  
    focal_gamma_max=3.75,  
    hard_negative_multiplier_min=1.0, # Hard negative mining range  
    hard_negative_multiplier_max=2.5,  
    epochs=18,                   # TCN/LSTM training epochs  
    batch_size=256,  
    hidden_size=64,  
    num_layers=2,
```

```

window_size=24,                # Default sequence window
dropout=0.20,
weight_decay=1e-4,
gradient_clip=1.0,
torch_fine_lr_scale=0.35,      # Fine-tuning LR multiplier
calibration_method='platt'     # Platt or isotonic
)

```

A.3 Walk-Forward Backtest Acceptance Gates

The six acceptance gates are parameterized as follows:

Table A1. Walk-forward backtest acceptance gates and their operational interpretations.

Gate	Metric	Threshold	Interpretation
1	min_trades_per_week	≥ 3.0	Sufficient signal frequency for statistical reliability
2	minimum_wfe	≥ 0.50	At least 50% of in-sample edge retained out-of-sample
3	max_single_trade_share	≤ 0.10	No single trade dominates total P&L
4	minimum_profitable_quarter_share	≥ 0.60	Profitable in 60%+ of quarterly periods
5	minimum_positive_composite_share	≥ 0.60	Positive P&L in 60%+ of composite regimes
6	net_pnl_total	> 0	Positive total net P&L after costs

A.4 Session Spread Cost Model

Table A2. Session-specific spread assumptions in the cost model.

Session Regime	Spread (pips)	Justification
overlap	1.0	London-NY overlap: highest liquidity, tightest spreads
london	1.5	Active European session: good liquidity
new_york	1.5	Active US session: good liquidity
asia	2.5	Reduced liquidity, wider spreads typical
off_hours	3.0	Weekend/early Asia: widest spreads, abstain policy active

A.5 Composite Regime Taxonomy

The 15 composite regime cells and their interpretations:

Table A3. Composite regime taxonomy with 15 cells from the cross-product of 5 trend states and 3 volatility states.

Trend State	Vol State	Composite Regime	Typical Characteristics
ranging	low	ranging_low	Compressed range, no clear direction
ranging	medium	ranging_medium	Moderate oscillation, no trend
ranging	high	ranging_high	Volatile chop, difficult to trade
weak_up	low	weak_up_low	Slow grind up, small ATR
weak_up	medium	weak_up_medium	Moderate trend, normal vol
weak_up	high	weak_up_high	Trend with spikes, watch for reversals
strong_up	low	strong_up_low	Steady uptrend, low volatility
strong_up	medium	strong_up_medium	Classic bull trend
strong_up	high	strong_up_high	Accelerating bull move
weak_down	low	weak_down_low	Slow grind down, small ATR
weak_down	medium	weak_down_medium	Moderate downtrend
weak_down	high	weak_down_high	Trend with spikes
strong_down	low	strong_down_low	Steady downtrend
strong_down	medium	strong_down_medium	Classic bear trend
strong_down	high	strong_down_high	Accelerating bear move

A.6 Feature Importance Top Features (Historical)

Based on historical SHAP analysis from classification model training runs documented in the system's training output files and the project README, the following features have consistently ranked among the top contributors across multiple training cycles:

Table A4. Top 15 historically important features from multi-model importance analysis.

Rank	Feature	Category	Description
1	Volume	Volume	Raw tick volume as primary signal
2	WILLR	Momentum	Williams %R (14-period)
3	STOCHRSI_K	Momentum	Stochastic RSI fast line
4	CCI	Momentum	Commodity Channel Index (14)
5	ADOSC	Volume	Accumulation/Distribution Oscillator
6	RSI	Momentum	Relative Strength Index (14)
7	CMO	Momentum	Chande Momentum Oscillator (14)
8	PLUS_DI	Trend	Positive Directional Indicator
9	MINUS_DI	Trend	Negative Directional Indicator
10	ADX	Trend	Average Directional Index

			(14)
11	HT_DCPHASE	Cycle	Hilbert Transform DC Phase
12	STOCHRSI_D	Momentum	Stochastic RSI slow line
13	BOP	Momentum	Balance of Power
14	MACD	Momentum	MACD line
15	rolling_std	Statistical	14-period rolling standard deviation

A.7 References

- Lopez de Prado, M. (2018). Advances in Financial Machine Learning. Wiley.
- Lin, T.Y., Goyal, P., Girshick, R., He, K., & Dollar, P. (2017). Focal Loss for Dense Object Detection. ICCV.
- Bai, S., Kolter, J.Z., & Koltun, V. (2018). An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv:1803.01271.
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD.
- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. KDD.
- Lundberg, S.M., & Lee, S.I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS.
- Prado, M.L. (2016). Building Diversified Portfolios that Outperform Out-of-Sample. Journal of Portfolio Management.